Maharashtra Education Society's
Institute of Management and Career Courses (IMCC), Pune
AUTONOMOUS

[CGPA-3.32] in Cycle 1

Academic Year 2026-27
FY MCA- SEM I
IT12 Data Structure and Algorithms
Practice Assignment1

Roll No: 2601103
Name: Tanvi Jagdish Khedekar

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None


class LinkedList:
    def __init__(self):
        self.head = None

    # 1. Create linked list
    def create_list(self, values):
        for value in values:
            self.insert_at_end(value)

    # Insert a node at the end
    def insert_at_end(self, data):
        new_node = Node(data)

        if self.head is None:
            self.head = new_node
        else:
            current_node = self.head

            while current_node.next is not None:
                current_node = current_node.next

            current_node.next = new_node

    # 2. Traverse and print the list
    def display(self):
        current_node = self.head

        while current_node is not None:
            print(current_node.data, end=" ")
```

```
                current_node = current_node.next

        print()

    # 3. Insert node at a specific position (0-based)
    def insert_at_position(self, data, position):
        new_node = Node(data)

        if position == 0:
            new_node.next = self.head
            self.head = new_node
            return

        current_node = self.head

        for _ in range(position - 1):
            if current_node is None:
                print("Invalid position")
                return
            current_node = current_node.next

        if current_node is None:
            print("Invalid position")
            return

        new_node.next = current_node.next
        current_node.next = new_node

    # 4. Find the middle node
    def find_middle(self):
        slow_node = self.head
        fast_node = self.head

        if self.head is None:
            print("List is empty")
            return

        while fast_node is not None and fast_node.next is not None:
            slow_node = slow_node.next
            fast_node = fast_node.next.next

        print("Middle node:", slow_node.data)

    # 5. Delete a node by value
    def delete_node(self, value):
        if self.head is None:
```

```
                return

        if self.head.data == value:
            self.head = self.head.next
            return

        current_node = self.head

        while current_node.next is not None:
            if current_node.next.data == value:
                current_node.next = current_node.next.next
                return

            current_node = current_node.next

    # 6. Reverse the linked list
    def reverse_list(self):
        previous_node = None
        current_node = self.head

        while current_node is not None:
            next_node = current_node.next
            current_node.next = previous_node
            previous_node = current_node
            current_node = next_node

        self.head = previous_node

    # 7. Sum every two consecutive nodes
    def sum_consecutive_nodes(self):
        current_node = self.head

        while current_node is not None and current_node.next is not None:
            total = current_node.data + current_node.next.data
            print(total, end=" ")
            current_node = current_node.next

        print()


# Main program
linked_list = LinkedList()

linked_list.create_list([10, 20, 30, 40])

print("Original list:")
```

ACCREDITED WITH GRADE A+ NAAC [CGPA-3.32] in Cycle 1

```
linked_list.display()

print("After inserting 25 at position 2:")
linked_list.insert_at_position(25, 2)
linked_list.display()

linked_list.find_middle()

print("After deleting 20:")
linked_list.delete_node(20)
linked_list.display()

print("Reversed list:")
linked_list.reverse_list()
linked_list.display()

print("Sum of consecutive nodes:")
linked_list.sum_consecutive_nodes()
```

OUTPUT:

```
PS D:\Projects\MCA\Semester_01\DataStructures_&_Algorithms\Practical_Work> python .\singlyLinkedList.py
Original list:
10 20 30 40
After inserting 25 at position 2:
10 20 25 30 40
Middle node: 25
After deleting 20:
10 25 30 40
Reversed list:
40 30 25 10
Sum of consecutive nodes:
70 55 35
```